

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Milestone 1 Documentation

Darapuneni Bhavya

TABLE OF CONTENTS

S.No	Title	Page No.
1	Project Description	1
2	Project Objectives	1
3	Quality Inspection Workflow	2
4	Dataset Used	2
5	Environment Setup	3
6	System Architecture	4
7	Frontend Development	5-10
8	Backend Development	11-14
9	Future Scope	15
10	Conclusion	15

1. Project Description

VisionInspect AI is a web-based application developed to support quality inspection in manufacturing industries. The system provides a centralized platform where users can securely log in, upload product images, and manage inspection-related activities through a simple and user-friendly interface. It is designed to streamline the inspection workflow and create a strong foundation for implementing intelligent quality inspection.

During **Milestone-1**, the project focused on setting up the complete application structure, including frontend and backend development, user authentication, image upload functionality, image preprocessing, dashboard development, and MVTec AD dataset integration. These modules establish the core system that will support AI-based defect detection and manufacturing analytics in the upcoming milestones.

2. Project Objectives

The primary objective of this project is to develop an AI-based **Manufacturing Defect Detection and Quality Inspection System** that automates the product inspection process using computer vision and machine learning techniques. The system aims to improve manufacturing quality by accurately detecting defects, reducing manual inspection effort, and enhancing production efficiency through intelligent quality inspection.

The objectives of the project are:

- Develop an automated quality inspection system for manufacturing products.
- Detect and identify product defects using AI and computer vision techniques.
- Classify detected defects and assess their severity for quality evaluation.
- Automate pass/fail quality decisions to support quality assurance processes.
- Minimize human errors during the inspection process.
- Reduce inspection time and improve overall production efficiency.
- Provide accurate inspection results through an interactive dashboard.
- Generate accurate inspection reports for effective quality monitoring.
- Improve overall product quality by identifying and reducing manufacturing defects.

3. Quality Inspection Workflow

The manufacturing quality inspection system follows a structured workflow for managing product images and preparing them for future AI-based inspection.

1. **User Authentication:** The user logs into the application using valid credentials to access the inspection system.
2. **Image Upload:** The user uploads a product image through the application interface for inspection.
3. **Image Validation:** The system validates the uploaded image by checking the file format and ensuring it is ready for further processing.
4. **Image Preprocessing:** The uploaded image is resized, converted to grayscale, and processed using basic image enhancement techniques to improve image quality and prepare it for future AI-based defect detection.
5. **Dashboard Management:** The dashboard displays uploaded images and inspection-related information, allowing users to monitor and manage inspection activities efficiently.
6. **Data Storage:** The uploaded image and related inspection information are temporarily managed by the backend to support future inspection, analysis, and AI-based defect detection.

4. Dataset Used

The **MVTec AD (Anomaly Detection) Dataset** is used in this project for manufacturing defect detection and quality inspection. It is a widely used benchmark dataset for industrial anomaly detection and contains high-quality images of different industrial products with both normal and defective samples. The dataset helps in developing and evaluating computer vision models for identifying manufacturing defects.

For **Milestone-1**, the MVTEC AD dataset was successfully downloaded, organized, and integrated into the project. It is used to support the image upload and preprocessing workflow, providing the required data for the inspection system. The dataset also establishes a strong foundation for implementing AI-based defect detection, defect classification, and quality assessment in the upcoming milestones.

Dataset Link : <https://www.mvtec.com/research-teaching/datasets/mvtec-ad>

5. Environment Setup

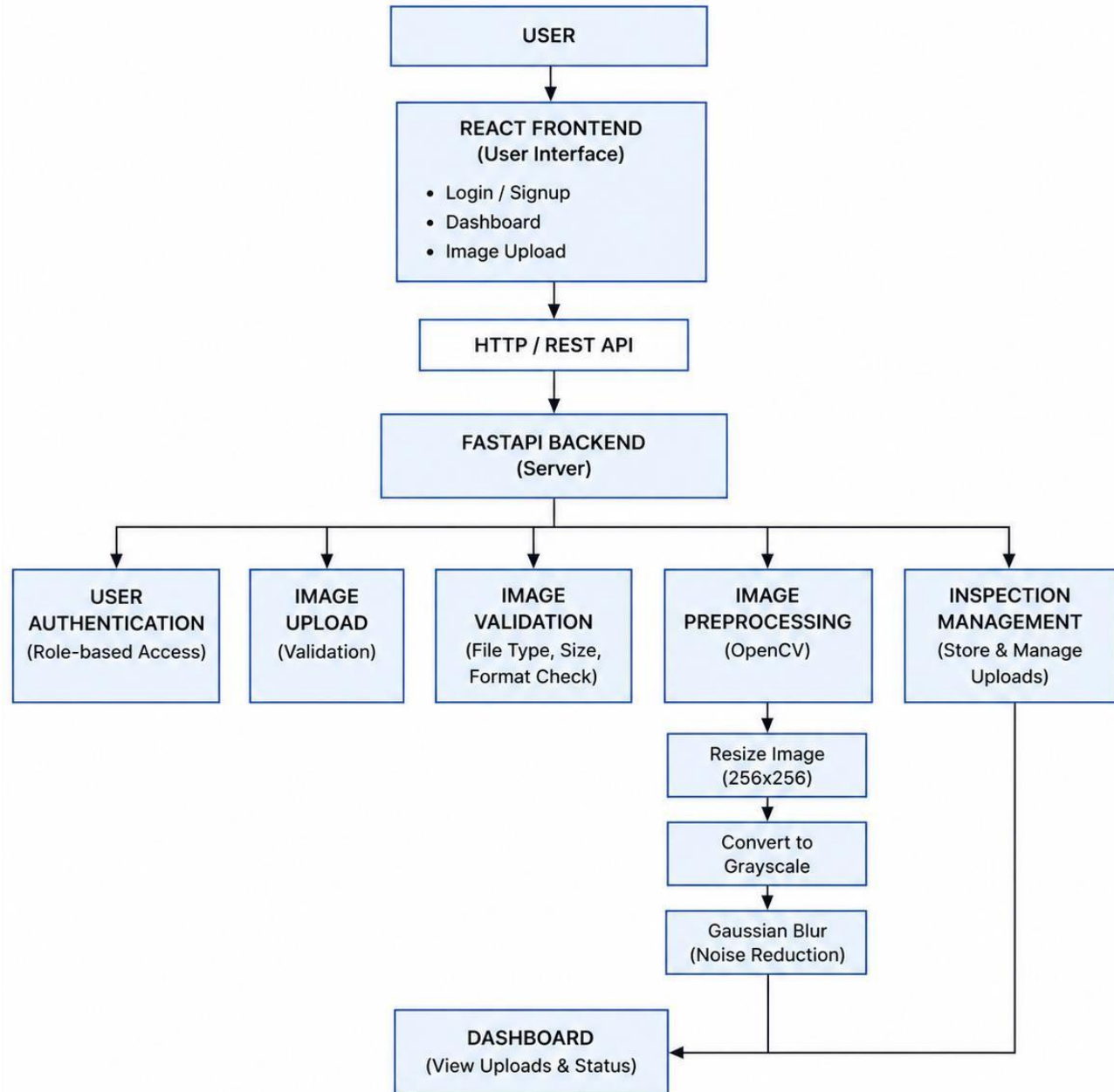
The development environment for **Manufacturing Defect Detection and Quality Inspection System** was configured to support both frontend and backend development. The project was developed using modern web technologies and Python-based frameworks to ensure efficient development and easy integration of future AI modules. All required software, libraries, and development tools were installed and configured before implementing the project modules.

The frontend was developed using **React.js**, **Vite**, and **Tailwind CSS** to create a responsive and user-friendly interface. The backend was built using **FastAPI**, which provides high-performance REST APIs for user authentication, image upload, and inspection management. **OpenCV** was used for image preprocessing operations such as resizing, grayscale conversion, and basic image enhancement. The project was developed using **Visual Studio Code**, while **Git** and **GitHub** were used for version control and project management. **Swagger UI** was used to test and verify the backend APIs during development.

Development Environment

Component	Technology Used
Frontend	React.js, Vite, Tailwind CSS
Backend	FastAPI
Image Processing	OpenCV
IDE	Visual Studio Code
Version Control	Git & GitHub
API Testing	Swagger UI

6. System Architecture



7. Frontend Development

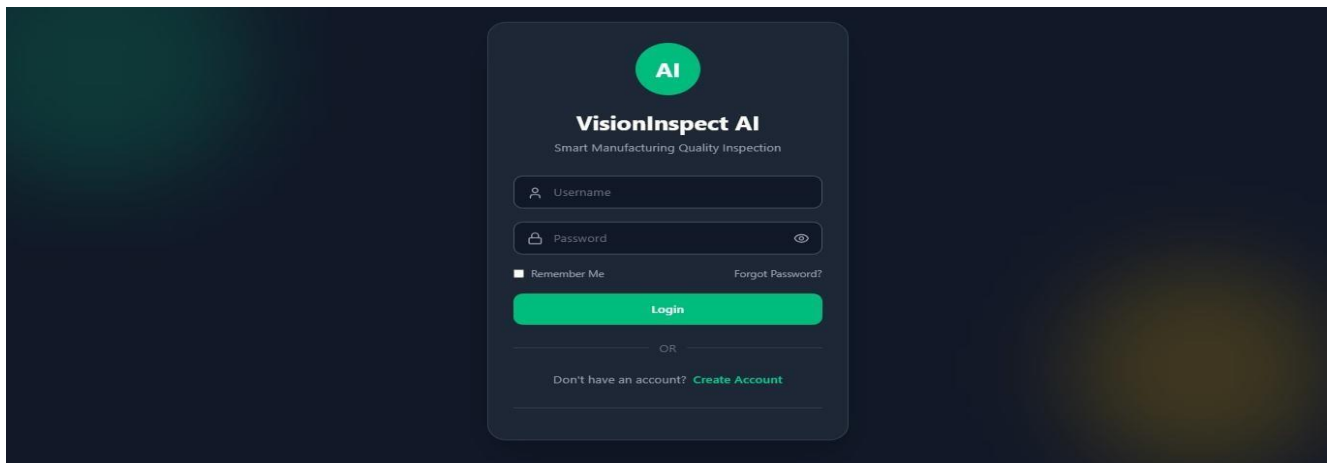
The frontend of the **Manufacturing Defect Detection & Quality Inspection System** was developed using **React.js**, **Vite**, and **Tailwind CSS** to provide a responsive and user-friendly interface. The frontend was designed with a modular architecture, allowing each functionality to be implemented as a separate component for better maintainability and scalability. React Router was used for page navigation, while Tailwind CSS was used to create a clean and responsive user interface.

During **Milestone-1**, the frontend implementation focused on developing the core modules required for the application, including user authentication pages, dashboard, image upload, and settings page. These modules communicate with the FastAPI backend through REST APIs, enabling users to perform inspection-related activities efficiently. The following frontend modules were successfully developed during this milestone.

Frontend Modules:

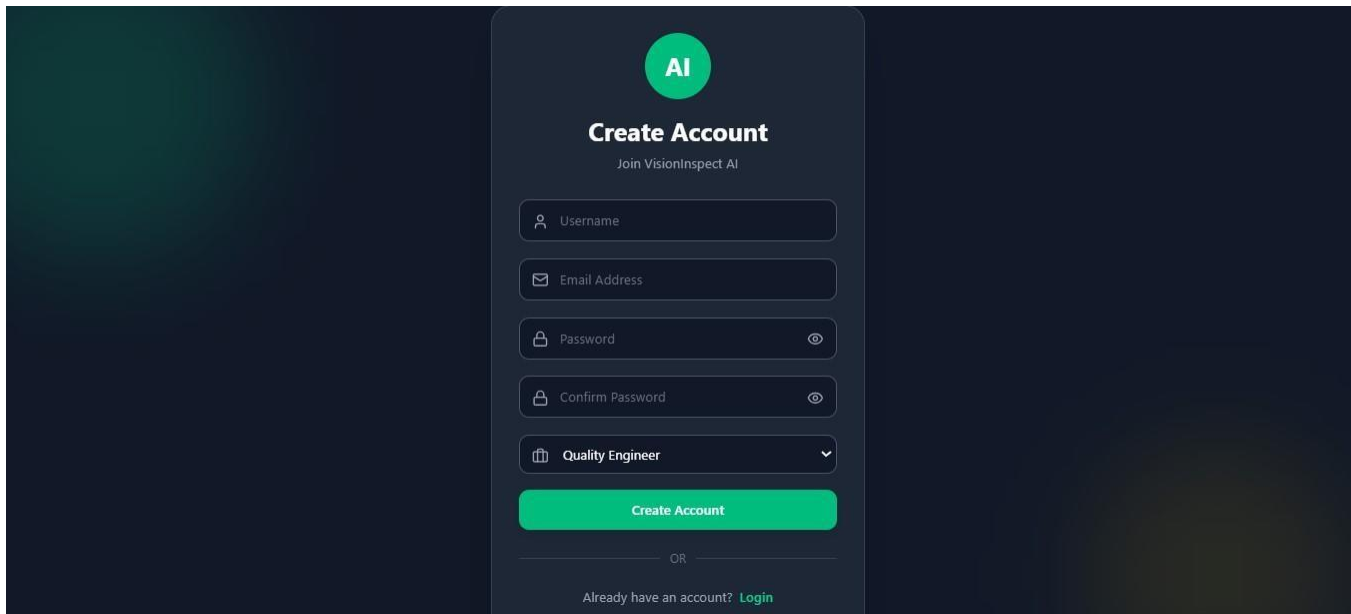
7.1 Login :

The **Login Module** allows registered users to securely access the application by entering their username and password. The login page validates the user credentials and communicates with the FastAPI backend for authentication. Based on the authentication result, users are granted access to the dashboard. This module serves as the entry point to the application and ensures that only authorized users can access the system.



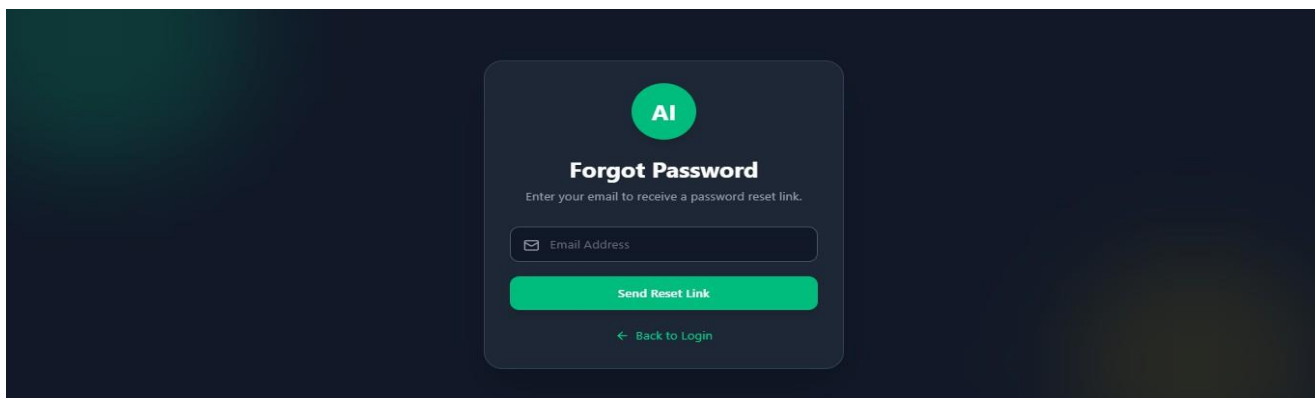
7.2 Signup :

The **Signup Module** enables new users to create an account by providing the required registration details. The module collects user information through a simple registration form and validates the entered data before submission. It provides a user-friendly interface for adding new users to the system and prepares them for future authentication.

A dark-themed user interface for creating an account. At the top, there is a green circular logo with the letters 'AI' in white. Below the logo, the text 'Create Account' is displayed in a bold, white font, followed by the subtitle 'Join VisionInspect AI' in a smaller, lighter font. The form consists of several input fields: 'Username' with a person icon, 'Email Address' with an envelope icon, 'Password' with a lock icon and a toggle eye icon, 'Confirm Password' with a lock icon and a toggle eye icon, and a dropdown menu for 'Quality Engineer' with a factory icon. A prominent green button labeled 'Create Account' is positioned below the input fields. At the bottom, there is a link that says 'Already have an account? Login'.

7.3 Forgot Password :

The **Forgot Password Module** allows users to recover their account if they forget their password. The module provides a simple interface where users can enter the required details to reset their password. During **Milestone-1**, a basic password recovery page was developed to improve user accessibility and enhance the overall user experience.

A dark-themed user interface for the forgot password process. It features a green circular logo with 'AI' at the top. The main heading is 'Forgot Password' in bold white text, with the instruction 'Enter your email to receive a password reset link.' below it. There is a single input field for 'Email Address' with an envelope icon. A green button labeled 'Send Reset Link' is located below the input field. At the bottom, there is a link that says '← Back to Login'.

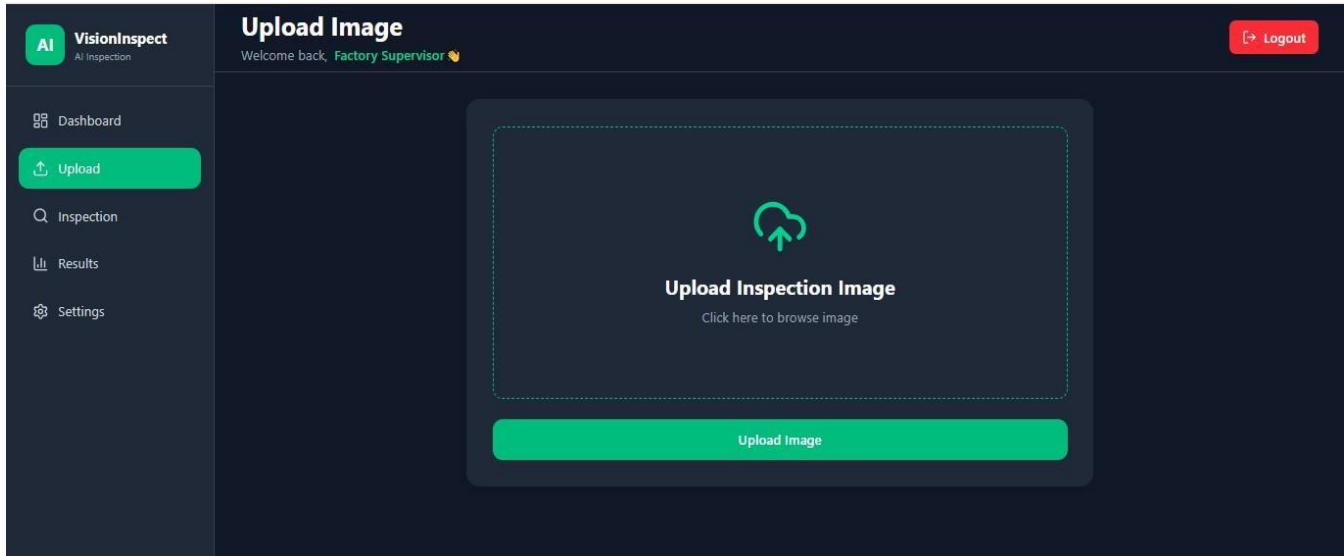
7.4 Dashboard :

The **Dashboard Module** serves as the main interface of the application after successful user login. It provides users with quick access to different features such as image upload, inspection management, and application settings. The dashboard was designed with a clean and responsive layout using Tailwind CSS, making navigation simple and user-friendly. During **Milestone-1**, the dashboard was developed to support the core workflow of the application.



7.5 Image Upload :

The **Image Upload Module** enables users to upload product images for inspection. The uploaded images are validated before being sent to the FastAPI backend for preprocessing. This module forms the initial step of the inspection workflow and prepares the images for future AI-based defect detection. During **Milestone-1**, the image upload functionality was successfully integrated with the backend using REST APIs.



7.6 Inspection :

The **Inspection Module** provides a centralized interface for viewing and managing inspection records. It displays the uploaded image details, inspection status, image size, and inspection date in a structured table format. This module allows users to monitor inspection activities efficiently and keeps track of previously processed images. During **Milestone-1**, the inspection page was developed to organize inspection records and support the overall quality inspection workflow.

Inspection History

Welcome back, **Factory Supervisor** 🧑‍🏭

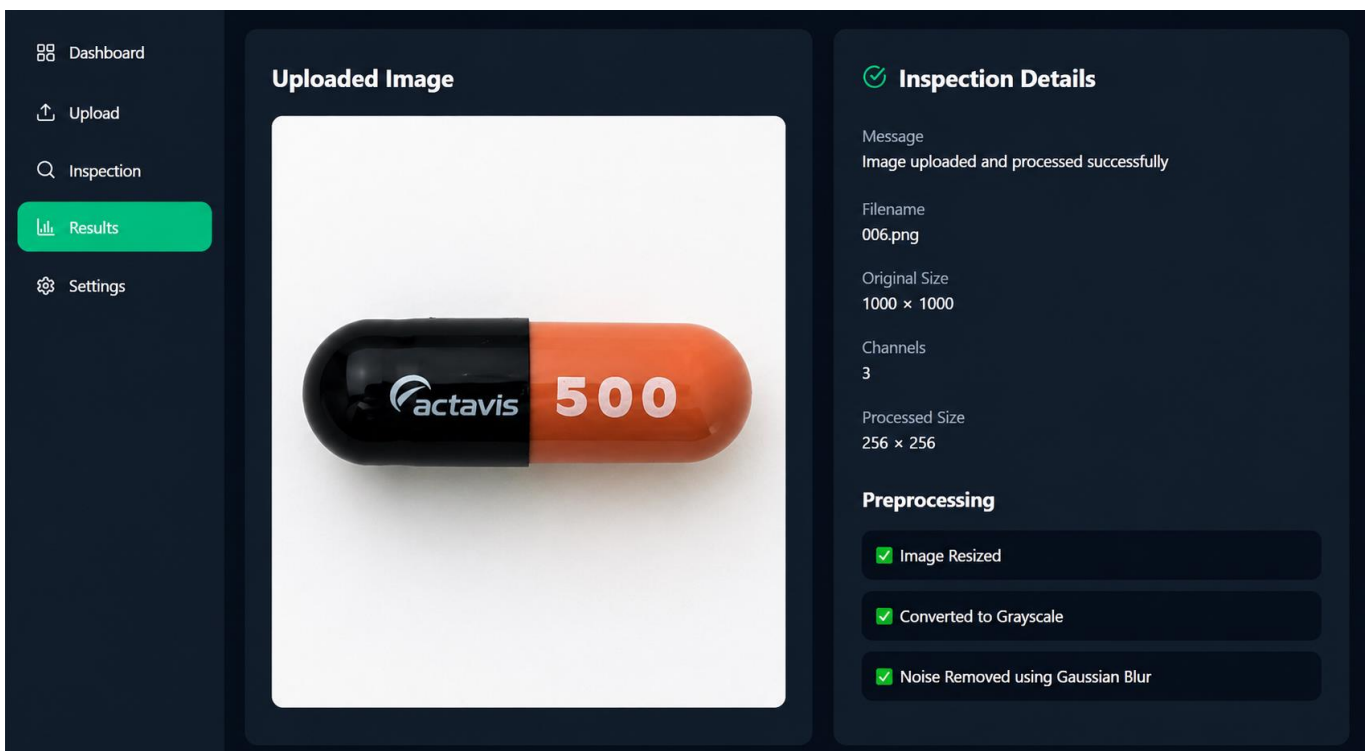
Filename	Status	Size	Date
001.png	Completed	1024 × 1024	7/10/2026, 5:13:59 PM
020.png	Completed	800 × 800	7/10/2026, 6:34:49 PM
006.png	Completed	1000 × 1000	7/10/2026, 6:36:05 PM

Logout

7.7 Results :

The **Results Module** displays the outcome of the uploaded image after it has been processed by the system. It provides a clear view of the uploaded image along with important inspection details such as the filename, original image size, processed image size, and the number of image channels. The module also displays the preprocessing operations performed on the uploaded image, including image resizing, grayscale conversion, and Gaussian blur for noise reduction.

During **Milestone-1**, the Results Module was developed to present the processed image and preprocessing details in a structured and user-friendly interface. This module allows users to verify that the uploaded image has been processed successfully and prepares the system for future AI-based defect detection and quality inspection.



7.8 Settings :

The **Settings Module** provides a dedicated page for managing user and application settings. It offers a structured interface for future customization and profile management features. During **Milestone-1**, the settings page was designed as part of the frontend to maintain a consistent user interface and support future enhancements.

The screenshot displays the Settings Module interface. On the left is a dark sidebar with navigation links: Dashboard, Upload, Inspection, Results, and Settings (highlighted in green). The main content area has a dark background and is divided into sections. The top section shows a user profile for 'admin' with email 'admin@gmail.com' and role 'Factory Supervisor'. Below this is the 'Account Information' section with fields for Username (admin), Email (admin@gmail.com), Role (Factory Supervisor), and Organization (VisionInspect AI). The bottom section is 'Security', featuring fields for Current Password, New Password, and Confirm Password, each with a placeholder text.

Account Information	
Username	admin
Email	admin@gmail.com
Role	Factory Supervisor
Organization	VisionInspect AI

Security		
Current Password	New Password	Confirm Password
Enter current password	Enter new password	Confirm new password

8. Backend Development

The backend of the **Manufacturing Defect Detection & Quality Inspection System** was developed using **FastAPI**, a modern Python framework for building high-performance REST APIs. The backend is responsible for handling user authentication, image upload, image validation, image preprocessing, and communication with the frontend. It provides the core functionality of the application by processing user requests and returning the required responses through REST APIs.

During **Milestone-1**, the backend was developed to support the frontend modules and implement the core inspection workflow. Image preprocessing operations such as image resizing, grayscale conversion, and Gaussian blur were implemented using **OpenCV**. The backend APIs were tested using **Swagger UI** to ensure proper communication between the frontend and backend. The modular backend architecture also provides a foundation for integrating AI-based defect detection and quality assessment in future milestones.



default		^
POST	/login Login	▼
POST	/inspect Inspect Image	▼
GET	/history Get History	▼
POST	/history Add History	▼
GET	/ Home	▼
POST	/upload Upload Image	▼

8.1 User Authentication :

The **User Authentication Module** verifies user credentials before allowing access to the application. During **Milestone-1**, authentication was implemented using predefined user credentials in the FastAPI backend. This module serves as the first layer of security and ensures that only authorized users can access the application.

POST /login Login

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "username": "admin",
  "password": "admin123"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "admin",
    "password": "admin123"
  }'
```

Request URL

http://127.0.0.1:8000/login

Server response

Code

Details

200

Response body

```
{
  "success": true,
  "username": "admin",
  "role": "Administratrarr",
  "message": "Login successful"
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 98
content-type: application/json
date: Fri, 28 Jun 2025 14:18:08 GMT
server: uvicorn
```

8.2 Image Upload and Preprocessing :

The **Image Upload and Preprocessing Module** is responsible for receiving product images from the frontend and preparing them for future AI-based defect detection. Users can upload an image through the application interface, which is then sent to the FastAPI backend using a REST API. The backend validates the uploaded image to ensure that it is in a supported format before initiating the preprocessing operations.

During **Milestone-1**, image preprocessing was implemented using **OpenCV**. After successful validation, the uploaded image is resized to **256 × 256** pixels, converted to **grayscale**, and processed using **Gaussian Blur** to reduce image noise. The backend returns the processed image details, including the filename, original image size, processed image size, and the preprocessing operations performed. This module establishes the foundation for integrating AI-based defect detection and quality inspection in the upcoming milestones.

POST /upload Upload Image

Parameters Cancel Reset

No parameters

Request body required multipart/form-data

file required Choose File 013.png

string

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/upload' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@013.png;type=Image/png'
```

Request URL

http://127.0.0.1:8000/upload

Server response

Code	Details
200	Response body <pre>{ "message": "Image uploaded and processed successfully", "filename": "013.png", "original_height": 1024, "original_width": 1024, "channels": 3, "processed_size": "256 x 256", "preprocessing": ["Image Resized", "Converted to Grayscale", "Noise Removed using Gaussian Blur"] }</pre> Download

8.3 Inspection History :

The **Inspection History Module** provides previously processed inspection records to the frontend through the **GET /history** API. The backend retrieves the stored inspection details and returns information such as the filename, inspection status, image dimensions, processed image size, and inspection date. This module enables users to view and monitor inspection records in a structured format through the Inspection History page.

The screenshot displays a REST client interface with the following sections:

- GET /history** Get History
- Parameters**: No parameters. A **Cancel** button is present.
- Execute** and **Clear** buttons.
- Responses**:
 - Curl**:

```
curl -X 'GET' \
'http://127.0.0.1:8080/history' \
-H 'accept: application/json'
```
 - Request URL**: `http://127.0.0.1:8080/history`
 - Server response**:

Code	Details
200	Response body <pre>[{ "filename": "001.png", "status": "Completed", "width": 1024, "height": 1024, "processed_size": "256 x 256", "date": "7/10/2026, 5:13:59 PM" }, { "filename": "020.png", "status": "Completed", "width": 800, "height": 800, "processed_size": "256 x 256", "date": "7/10/2026, 6:34:49 PM" }, { "filename": "006.png", "status": "Completed", "width": 1000, "height": 1000, "processed_size": "256 x 256", "date": "7/10/2026, 6:36:05 PM" }]</pre> Download

9. Future Scope

The **Manufacturing Defect Detection & Quality Inspection System** provides a strong foundation for implementing advanced AI-based quality inspection features in future milestones. The following enhancements are planned to improve the functionality of the system:

- Integrate AI models for automatic defect detection.
- Implement defect classification and quality assessment.
- Store inspection records using a database.
- Generate detailed inspection reports.
- Enhance dashboard analytics and visualization.
- Improve user management and application features.

10. Conclusion

The **Manufacturing Defect Detection & Quality Inspection System** was successfully developed up to **Milestone-1**, focusing on the implementation of the core frontend and backend functionalities. The system includes user authentication, image upload, image preprocessing, inspection history, and results visualization through a responsive user interface. The completed work establishes a strong foundation for implementing AI-based defect detection and advanced quality inspection features in the upcoming milestones.